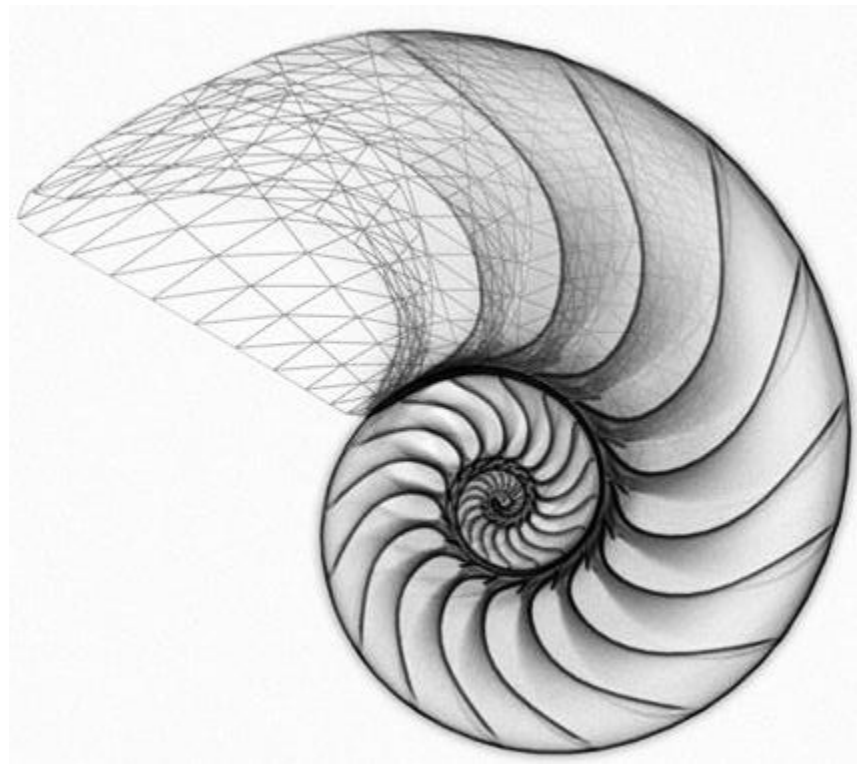




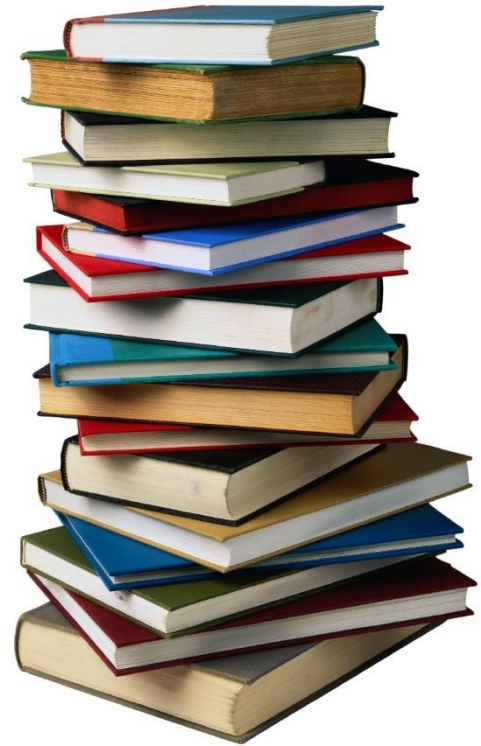
Lecture 07

Abstract Data Type Stack and Queue (Array-based Implementation)

CSE225: Data Structures and Algorithms

Stack

- A list
- Data items can be added and deleted
- Maintains **Last In First Out (LIFO)** order



Specification of **StackType**

Structure:	Elements are added to and removed from the top of the stack.
Definitions (provided by user):	
MAX_ITEMS	Maximum number of items that might be on the stack.
ItemType	Data type of the items on the stack.
Operations (provided by the ADT):	
MakeEmpty	
Function	Sets stack to an empty state.
Postcondition	Stack is empty.
Boolean IsEmpty	
Function	Determines whether the stack is empty.
Precondition	Stack has been initialized.
Postcondition	Returns true if stack is empty and false otherwise.
Boolean IsFull	
Function	Determines whether the stack is full.
Precondition	Stack has been initialized.
Postcondition	Returns true if stack is full and false otherwise.

Specification of **StackType**

Push(ItemType newItem)	
Function	Adds newItem to the top of the stack.
Precondition	Stack has been initialized.
Postcondition	If (stack is full), exception FullStack is thrown, else newItem is at the top of the stack.
Pop()	
Function	Removes top item from the stack.
Precondition	Stack has been initialized.
Postcondition	If (stack is empty), exception EmptyStack is thrown, else top element has been removed from stack.
ItemType Top()	
Function	Returns a copy of the top item on the stack.
Precondition	Stack has been initialized.
Postcondition	If (stack is empty), exception EmptyStack is thrown, else a copy of the top element is returned.

stacktype.h

```
#ifndef STACKTYPE_H_INCLUDED
#define STACKTYPE_H_INCLUDED

const int MAX_ITEMS = 5;

class FullStack
{}; // Exception class thrown by Push when stack is full.
class EmptyStack
{}; // Exception class thrown by Pop and Top when stack is empty.

template <class ItemType>
class StackType
{
    public:
        StackType();
        bool IsFull();
        bool IsEmpty();
        void MakeEmpty();
        void Push(ItemType);
        void Pop();
        ItemType Top();
    private:
        int top;
        ItemType items[MAX_ITEMS];
};

#endif // STACKTYPE_H_INCLUDED
```

stacktype.cpp

```
#include "StackType.h"
template <class ItemType>

StackType<ItemType>::StackType()
{
    top = -1;
}
template <class ItemType>
bool StackType<ItemType>::IsEmpty()
{
    return (top == -1);
}
void StackType<ItemType>::MakeEmpty()
{
    top = -1;
}
template <class ItemType>
bool StackType<ItemType>::IsFull()
{
    return (top == MAX_ITEMS-1);
}
```

```
template <class ItemType>
void StackType<ItemType>::Push(ItemType
newItem)
{
    if( IsFull() )
        throw FullStack();
    top++;
    items[top] = newItem;
}
template <class ItemType>
void StackType<ItemType>::Pop()
{
    if( IsEmpty() )
        throw EmptyStack();
    top--;
}
template <class ItemType>
ItemType StackType<ItemType>::Top()
{
    if (IsEmpty())
        throw EmptyStack();
    return items[top];
}
```

stacktype.cpp

```
#include "StackType.h"
template <class ItemType>

StackType<ItemType>::StackType()
{
    top = -1;
}
template <class ItemType>
bool StackType<ItemType>::IsEmpty()
{
    return (top == -1);
}
void StackType<ItemType>::MakeEmpty()
{
    top = -1;
}
template <class ItemType>
bool StackType<ItemType>::IsFull()
{
    return (top == MAX_ITEMS-1);
}
```

O(1)

O(1)

O(1)

O(1)

```
template <class ItemType>
void StackType<ItemType>::Push(ItemType
newItem)
{
    if( IsFull() )
        throw FullStack();
    top++;
    items[top] = newItem;
}
template <class ItemType>
void StackType<ItemType>::Pop()
{
    if( IsEmpty() )
        throw EmptyStack();
    top--;
}
template <class ItemType>
ItemType StackType<ItemType>::Top()
{
    if (IsEmpty())
        throw EmptyStack();
    return items[top];
}
```

O(1)

O(1)

O(1)

Application of Stack

()

(()) () (() ()) ()

(()) () ((()

(()))) ((()

Which of the strings of parentheses are balanced?

Application of Stack

Algorithm for matching parentheses string

1. Initialise an empty stack
2. Read next item in the string
 - a) If item is an opening parentheses, push it into the stack
 - b) Else, if item is a closing parentheses, pop from stack
3. If there are more items to process, go to step 2
4. Pop the answer off the stack.

Application of Stack

(()) () (() ()) ()

Application of Stack

(()) () (() ()) ()

string

(	(	)	)	(	)	(	(	)	(	)	)	(	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---

stack

--	--	--	--	--	--	--	--	--	--

Application of Stack

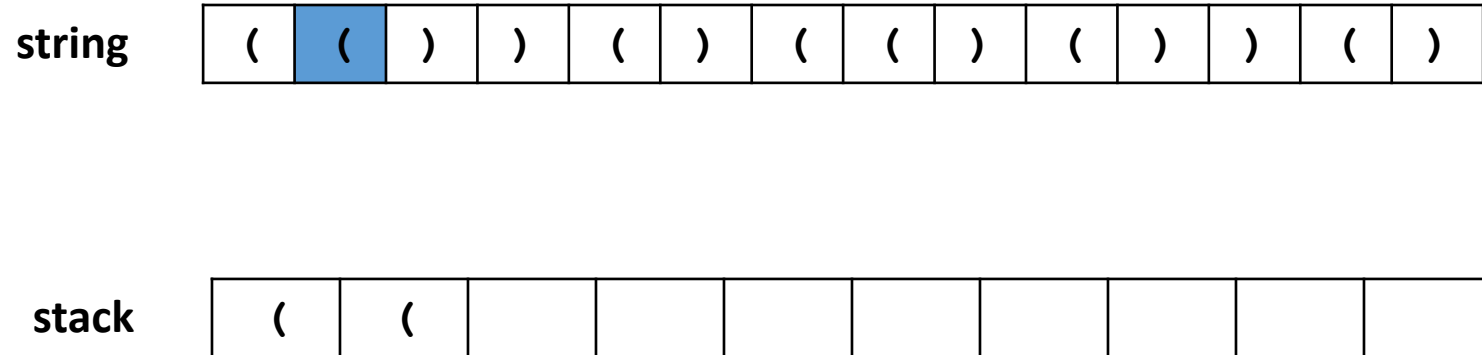
string

(	(	)	)	(	)	(	(	)	(	)	)	(	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---

stack

(									
---	--	--	--	--	--	--	--	--	--

Application of Stack



Application of Stack

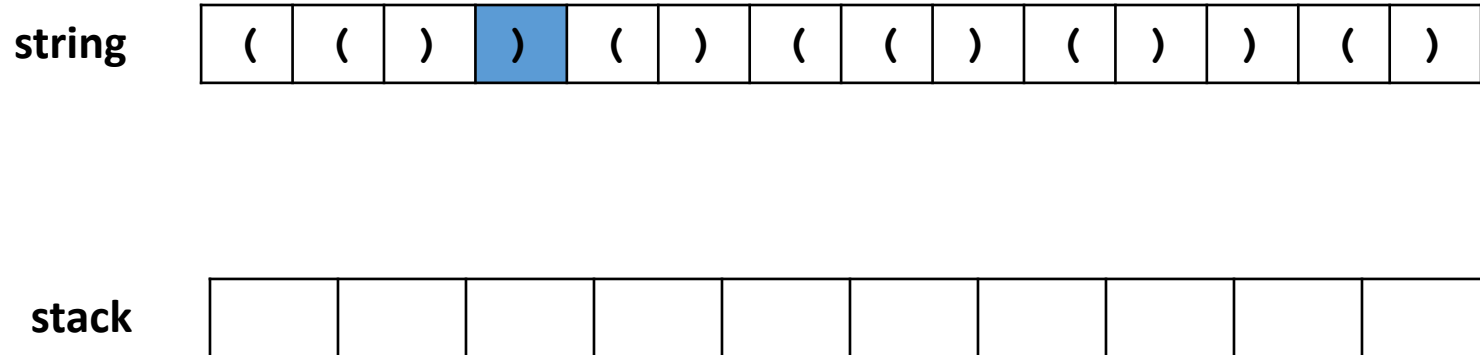
string

(	(	)	)	(	)	(	(	)	(	)	)	(	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---

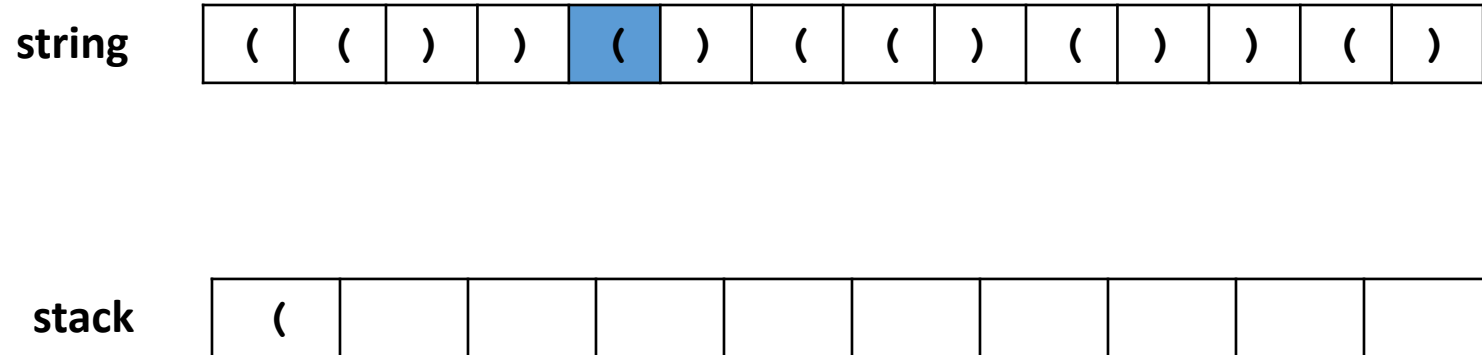
stack

(									
---	--	--	--	--	--	--	--	--	--

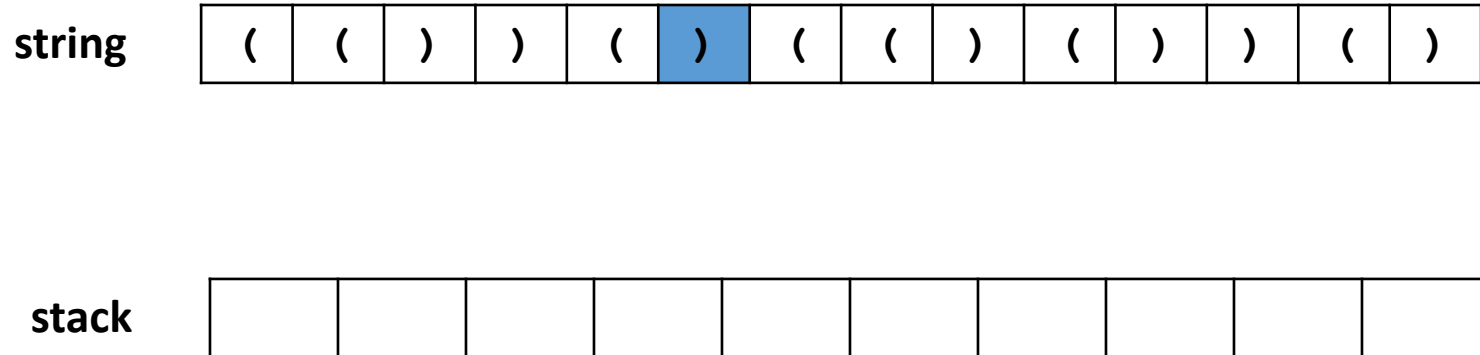
Application of Stack



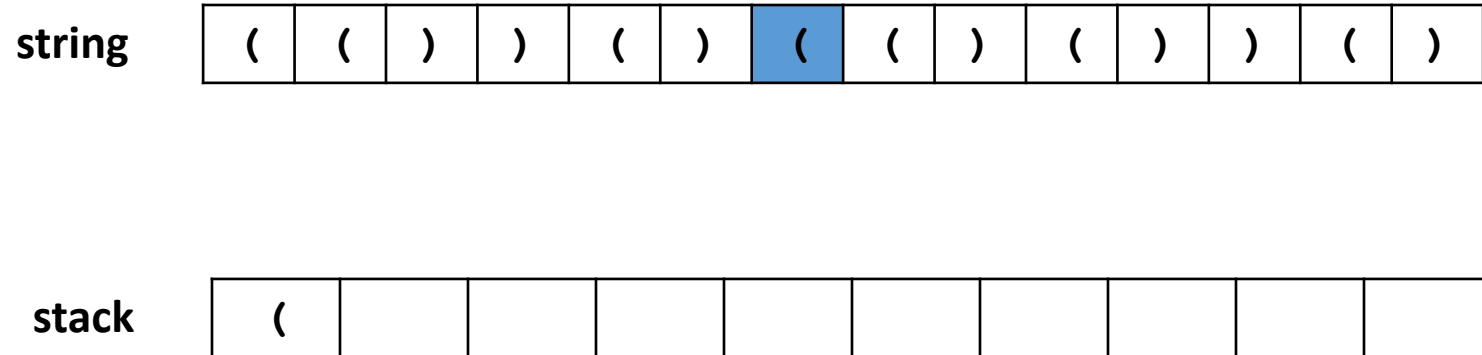
Application of Stack



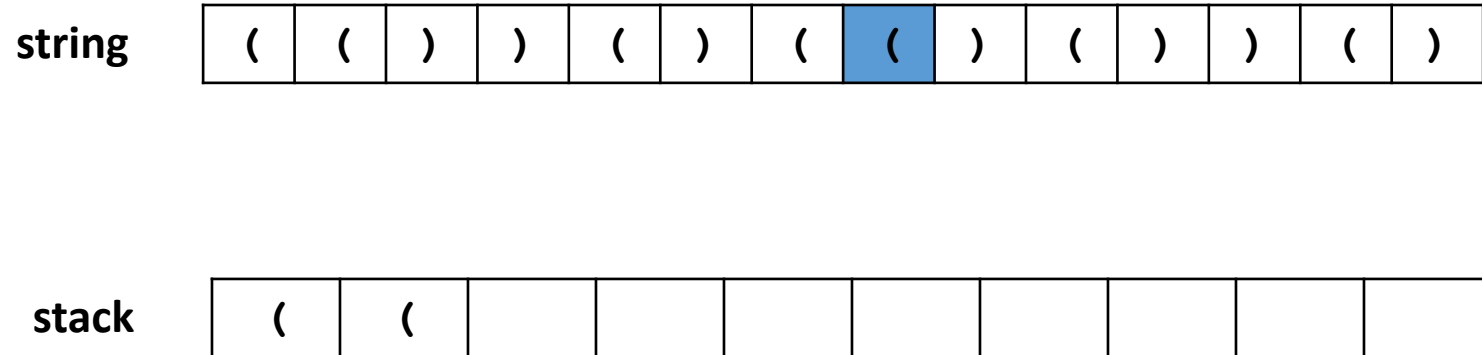
Application of Stack



Application of Stack

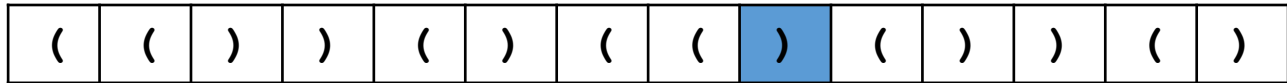


Application of Stack

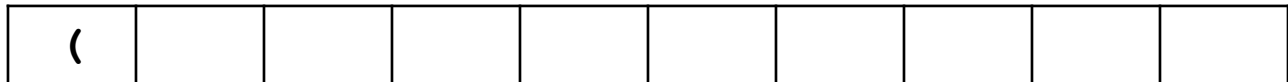


Application of Stack

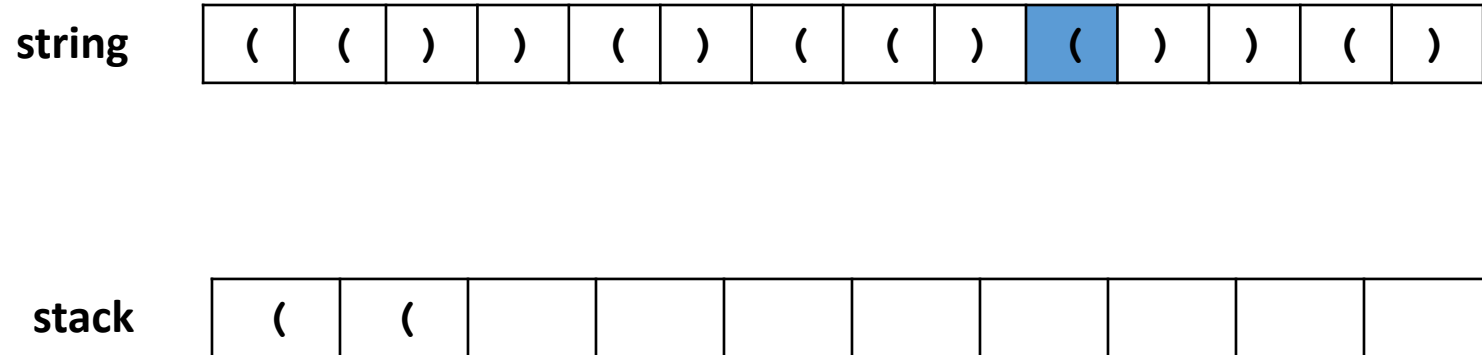
string



stack

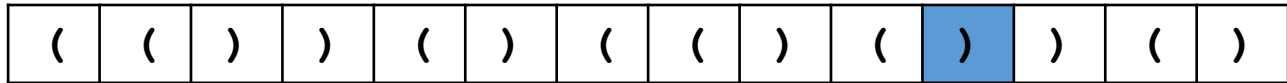


Application of Stack

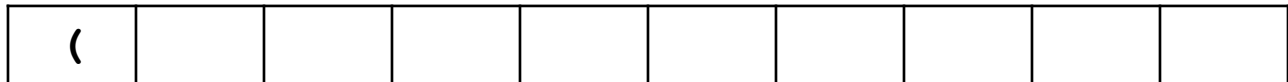


Application of Stack

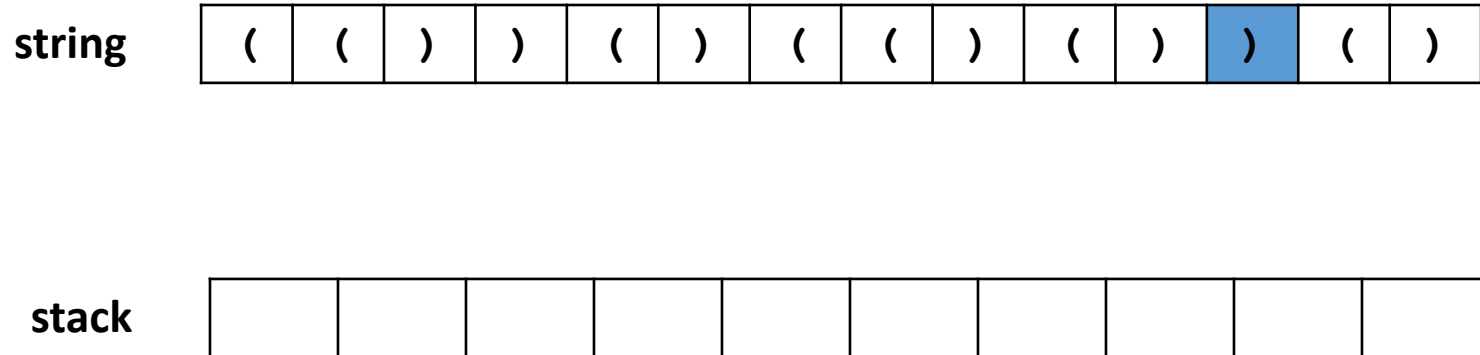
string



stack

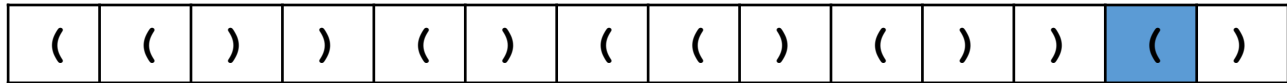


Application of Stack

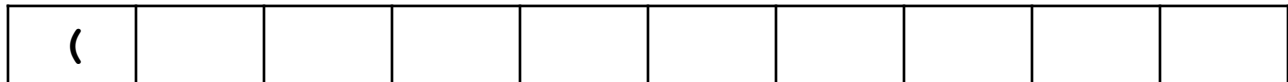


Application of Stack

string



stack



Application of Stack

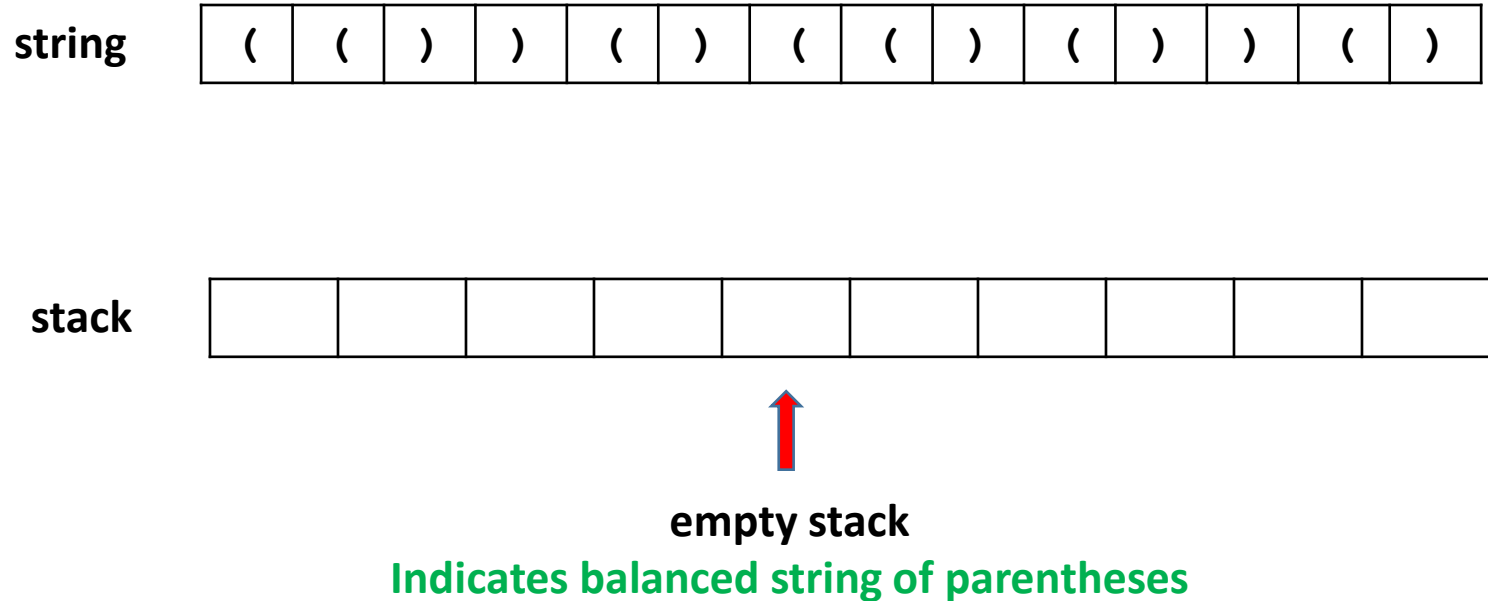
string

(	(	)	)	(	)	(	(	)	(	)	)	(	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---

stack

[illegible]

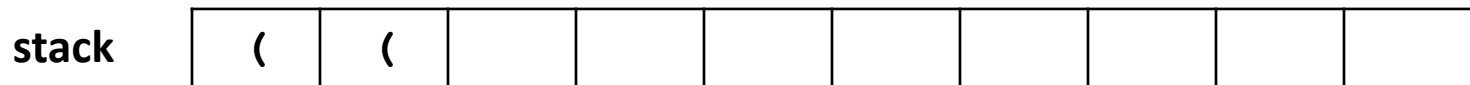
Application of Stack



Application of Stack

(()) () (())

Consider this string. After processing each item, the stack is not empty.



non-empty stack

Indicates unbalanced string of parentheses

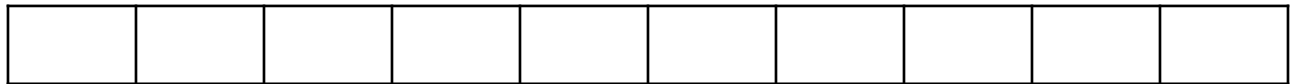
Application of Stack

(())) (()



Consider this string. When processing indicated item, you are trying to pop from empty stack.

stack



unsuccessful pop

Indicates unbalanced string of parentheses

Application of Stack

Evaluating arithmetic expressions

Infix	Postfix	Evaluation
$2 - 3 * 4 + 5$	$2\ 3\ 4\ *\ -\ 5\ +$	-5
$(2 - 3) * (4 + 5)$	$2\ 3\ -\ 4\ 5\ +\ *$	-9
$2 - (3 * 4 + 5)$	$2\ 3\ 4\ *\ 5\ +\ -$	-15

Why ? No parentheses necessary !

Application of Stack

Algorithm for Infix to Postfix

1. Examine the next element in the input.
2. If it is **operand**, output it.
3. If it is **opening parenthesis**, push it on stack.
4. If it is an **operator**, then
 - Pop until the top of the stack has an element of lower precedence
 - Then push it
5. If it is a **closing parenthesis**, pop operators from stack and output them until an opening parenthesis is encountered. pop and discard the opening parenthesis.
6. If there is **more input** go to step 1
7. If there is **no more input**, **pop** the remaining operators to output.

Application of Stack

Suppose we want to convert

$(4+8) * (6-5) / ((3-2) * (2+2))$ into Postfix form

Application of Stack

Infix

(	4	+	8	)	*	(	6	-	5	)	/	(	(	3	-	2	)	*	(	2	+	2	)	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stack

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Postfix

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Application of Stack

Infix

(	4	+	8	)	*	(	6	-	5	)	/	(	(	3	-	2	)	*	(	2	+	2	)	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stack

(															
---	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Postfix

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Application of Stack

Infix

(	4	+	8	)	*	(	6	-	5	)	/	(	(	3	-	2	)	*	(	2	+	2	)	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stack

(														
---	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Postfix

4														
---	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Application of Stack

Infix

(	4	+	8	)	*	(	6	-	5	)	/	(	(	3	-	2	)	*	(	2	+	2	)	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stack

(	+													
---	---	--	--	--	--	--	--	--	--	--	--	--	--	--

Postfix

4														
---	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Application of Stack

Infix

(	4	+	8	)	*	(	6	-	5	)	/	(	(	3	-	2	)	*	(	2	+	2	)	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stack

(	+													
---	---	--	--	--	--	--	--	--	--	--	--	--	--	--

Postfix

4	8													
---	---	--	--	--	--	--	--	--	--	--	--	--	--	--

Application of Stack

Infix

(	4	+	8	)	*	(	6	-	5	)	/	(	(	3	-	2	)	*	(	2	+	2	)	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stack

[illegible]

Postfix

[illegible]

Application of Stack

Infix

(	4	+	8	)	*	(	6	-	5	)	/	(	(	3	-	2	)	*	(	2	+	2	)	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stack

*														
---	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Postfix

4	8	+												
---	---	---	--	--	--	--	--	--	--	--	--	--	--	--

Application of Stack

Infix

(	4	+	8	)	*	(	6	-	5	)	/	(	(	3	-	2	)	*	(	2	+	2	)	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stack

*	(																
---	---	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Postfix

4	8	+															
---	---	---	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Application of Stack

Infix

(	4	+	8	)	*	(	6	-	5	)	/	(	(	3	-	2	)	*	(	2	+	2	)	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stack

*	(																
---	---	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Postfix

4	8	+	6														
---	---	---	---	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Application of Stack

Infix

(	4	+	8	)	*	(	6	-	5	)	/	(	(	3	-	2	)	*	(	2	+	2	)	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stack

*	(	-												
---	---	---	--	--	--	--	--	--	--	--	--	--	--	--

Postfix

4	8	+	6											
---	---	---	---	--	--	--	--	--	--	--	--	--	--	--

Application of Stack

Infix

(	4	+	8	)	*	(	6	-	5	)	/	(	(	3	-	2	)	*	(	2	+	2	)	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stack

*	(	-												
---	---	---	--	--	--	--	--	--	--	--	--	--	--	--

Postfix

4	8	+	6	5										
---	---	---	---	---	--	--	--	--	--	--	--	--	--	--

Application of Stack

Infix

(	4	+	8	)	*	(	6	-	5	)	/	(	(	3	-	2	)	*	(	2	+	2	)	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stack

*																								
---	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Postfix

4	8	+	6	5	-																			
---	---	---	---	---	---	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Application of Stack

Infix

(	4	+	8	)	*	(	6	-	5	)	/	(	(	3	-	2	)	*	(	2	+	2	)	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stack

/														
---	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Postfix

4	8	+	6	5	-	*								
---	---	---	---	---	---	---	--	--	--	--	--	--	--	--

Application of Stack

Infix

(	4	+	8	)	*	(	6	-	5	)	/	(	(	3	-	2	)	*	(	2	+	2	)	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stack

/	(													
---	---	--	--	--	--	--	--	--	--	--	--	--	--	--

Postfix

4	8	+	6	5	-	*								
---	---	---	---	---	---	---	--	--	--	--	--	--	--	--

Application of Stack

Infix

(	4	+	8	)	*	(	6	-	5	)	/	(	(	3	-	2	)	*	(	2	+	2	)	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stack

/	(	(															
---	---	---	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Postfix

4	8	+	6	5	-	*											
---	---	---	---	---	---	---	--	--	--	--	--	--	--	--	--	--	--

Application of Stack

Infix

(	4	+	8	)	*	(	6	-	5	)	/	(	(	3	-	2	)	*	(	2	+	2	)	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stack

/	(	(												
---	---	---	--	--	--	--	--	--	--	--	--	--	--	--

Postfix

4	8	+	6	5	-	*	3							
---	---	---	---	---	---	---	---	--	--	--	--	--	--	--

Application of Stack

Infix

(	4	+	8	)	*	(	6	-	5	)	/	(	(	3	-	2	)	*	(	2	+	2	)	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stack

/	(	(	-											
---	---	---	---	--	--	--	--	--	--	--	--	--	--	--

Postfix

4	8	+	6	5	-	*	3							
---	---	---	---	---	---	---	---	--	--	--	--	--	--	--

Application of Stack

Infix

(	4	+	8	)	*	(	6	-	5	)	/	(	(	3	-	2	)	*	(	2	+	2	)	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stack

/	(	(	-											
---	---	---	---	--	--	--	--	--	--	--	--	--	--	--

Postfix

4	8	+	6	5	-	*	3	2						
---	---	---	---	---	---	---	---	---	--	--	--	--	--	--

Application of Stack

Infix

(	4	+	8	)	*	(	6	-	5	)	/	(	(	3	-	2	)	*	(	2	+	2	)	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stack

/	(																
---	---	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Postfix

4	8	+	6	5	-	*	3	2	-					
---	---	---	---	---	---	---	---	---	---	--	--	--	--	--

Application of Stack

Infix

(	4	+	8	)	*	(	6	-	5	)	/	(	(	3	-	2	)	*	(	2	+	2	)	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stack

/	(	*												
---	---	---	--	--	--	--	--	--	--	--	--	--	--	--

Postfix

4	8	+	6	5	-	*	3	2	-					
---	---	---	---	---	---	---	---	---	---	--	--	--	--	--

Application of Stack

Infix

(	4	+	8	)	*	(	6	-	5	)	/	(	(	3	-	2	)	*	(	2	+	2	)	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stack

/	(	*	(														
---	---	---	---	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Postfix

4	8	+	6	5	-	*	3	2	-								
---	---	---	---	---	---	---	---	---	---	--	--	--	--	--	--	--	--

Application of Stack

Infix

(	4	+	8	)	*	(	6	-	5	)	/	(	(	3	-	2	)	*	(	2	+	2	)	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stack

/	(	*	(											
---	---	---	---	--	--	--	--	--	--	--	--	--	--	--

Postfix

4	8	+	6	5	-	*	3	2	-	2				
---	---	---	---	---	---	---	---	---	---	---	--	--	--	--

Application of Stack

Infix

(	4	+	8	)	*	(	6	-	5	)	/	(	(	3	-	2	)	*	(	2	+	2	)	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stack

/	(	*	(	+										
---	---	---	---	---	--	--	--	--	--	--	--	--	--	--

Postfix

4	8	+	6	5	-	*	3	2	-	2				
---	---	---	---	---	---	---	---	---	---	---	--	--	--	--

Application of Stack

Infix

(	4	+	8	)	*	(	6	-	5	)	/	(	(	3	-	2	)	*	(	2	+	2	)	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stack

/	(	*	(	+										
---	---	---	---	---	--	--	--	--	--	--	--	--	--	--

Postfix

4	8	+	6	5	-	*	3	2	-	2	2			
---	---	---	---	---	---	---	---	---	---	---	---	--	--	--

Application of Stack

Infix

(	4	+	8	)	*	(	6	-	5	)	/	(	(	3	-	2	)	*	(	2	+	2	)	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stack

/	(	*												
---	---	---	--	--	--	--	--	--	--	--	--	--	--	--

Postfix

4	8	+	6	5	-	*	3	2	-	2	2	+		
---	---	---	---	---	---	---	---	---	---	---	---	---	--	--

Application of Stack

Infix

(	4	+	8	)	*	(	6	-	5	)	/	(	(	3	-	2	)	*	(	2	+	2	)	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stack

/														
---	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Postfix

4	8	+	6	5	-	*	3	2	-	2	2	+	*	
---	---	---	---	---	---	---	---	---	---	---	---	---	---	--

Application of Stack

Infix

(	4	+	8	)	*	(	6	-	5	)	/	(	(	3	-	2	)	*	(	2	+	2	)	)
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stack

[illegible]

Postfix

4	8	+	6	5	-	*	3	2	-	2	2	+	*	/
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Application of Stack

Now let's evaluate this expression.

4 8 + 6 5 - * 3 2 - 2 2 + * /

Application of Stack

Algorithm for evaluating a postfix expression

1. Initialise an empty stack
2. Read next item in the expression
 - a) If item is an operand, push it into the stack
 - b) Else, if item is an operator, pop top two items off the stack, apply the operator, and push the answer back into the stack
3. If there are more items to process, go to step 2
4. Pop the answer off the stack.

Application of Stack

string

4	8	+	6	5	-	*	3	2	-	2	2	+	*	/
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

stack

--	--	--	--	--	--	--	--	--	--

Application of Stack

string

4	8	+	6	5	-	*	3	2	-	2	2	+	*	/
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

stack

4									
---	--	--	--	--	--	--	--	--	--

Application of Stack

string

4	8	+	6	5	-	*	3	2	-	2	2	+	*	/
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

stack

4	8								
---	---	--	--	--	--	--	--	--	--

Application of Stack

string

4	8	+	6	5	-	*	3	2	-	2	2	+	*	/
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

stack

12									
----	--	--	--	--	--	--	--	--	--

Application of Stack

string

4	8	+	6	5	-	*	3	2	-	2	2	+	*	/
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

stack

12	6								
----	---	--	--	--	--	--	--	--	--

Application of Stack

string

4	8	+	6	5	-	*	3	2	-	2	2	+	*	/
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

stack

12	6	5							
----	---	---	--	--	--	--	--	--	--

Application of Stack

string

4	8	+	6	5	-	*	3	2	-	2	2	+	*	/
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

stack

12	1								
----	---	--	--	--	--	--	--	--	--

Application of Stack

string

4	8	+	6	5	-	*	3	2	-	2	2	+	*	/
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

stack

12									
----	--	--	--	--	--	--	--	--	--

Application of Stack

string

4	8	+	6	5	-	*	3	2	-	2	2	+	*	/
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

stack

12	3								
----	---	--	--	--	--	--	--	--	--

Application of Stack

string

4	8	+	6	5	-	*	3	2	-	2	2	+	*	/
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

stack

12	3	2							
----	---	---	--	--	--	--	--	--	--

Application of Stack

string

4	8	+	6	5	-	*	3	2	-	2	2	+	*	/
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

stack

12	1								
----	---	--	--	--	--	--	--	--	--

Application of Stack

string

4	8	+	6	5	-	*	3	2	-	2	2	+	*	/
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

stack

12	1	2							
----	---	---	--	--	--	--	--	--	--

Application of Stack

string

4	8	+	6	5	-	*	3	2	-	2	2	+	*	/
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

stack

12	1	2	2						
----	---	---	---	--	--	--	--	--	--

Application of Stack

string

4	8	+	6	5	-	*	3	2	-	2	2	+	*	/
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

stack

12	1	4							
----	---	---	--	--	--	--	--	--	--

Application of Stack

string

4	8	+	6	5	-	*	3	2	-	2	2	+	*	/
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

stack

12	4								
----	---	--	--	--	--	--	--	--	--

Application of Stack

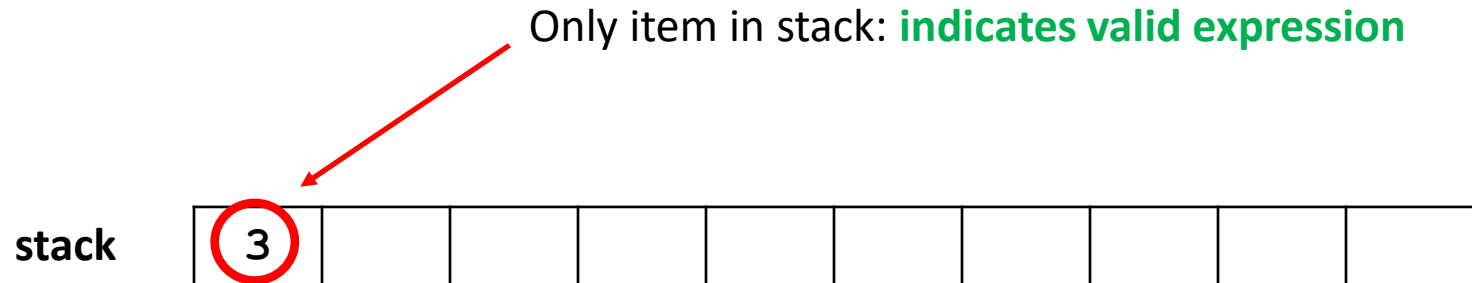
string

4	8	+	6	5	-	*	3	2	-	2	2	+	*	/
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

stack

3									
---	--	--	--	--	--	--	--	--	--

Application of Stack



More than one item in stack or unsuccessful pop:
indicates invalid expression

Queue

- A list
- Data items can be added and deleted
- Maintains **First In First Out (FIFO)** order



Specification of QueueType

Structure:	Elements are added to the rear and removed from the front of the queue.
Definitions (provided by user):	
MAX_ITEMS	Maximum number of items that might be on the queue.
ItemType	Data type of the items on the queue.
Operations (provided by the ADT):	
MakeEmpty	
Function	Sets stack to an empty state.
Postcondition	Stack is empty.
Boolean IsEmpty	
Function	Determines whether the stack is empty.
Precondition	Stack has been initialized.
Postcondition	Returns true if stack is empty and false otherwise.
Boolean IsFull	
Function	Determines whether the stack is full.
Precondition	Stack has been initialized.
Postcondition	Returns true if stack is full and false otherwise.

Specification of **QueueType**

Enqueue(ItemType newItem)	
Function	Adds newItem to the rear of the queue.
Precondition	Queue has been initialized.
Postcondition	If (queue is full), FullQueue exception is thrown, else newItem is at rear of queue.
Dequeue(ItemType& item)	
Function	Removes front item from the queue and returns it in item.
Precondition	Queue has been initialized.
Postcondition	If (queue is empty), EmptyQueue exception is thrown and item is undefined, else front element has been removed from queue and item is a copy of removed element.

Implementation Issues

- Always insert elements at the back of the array.



Implementation Issues

- Maintain two indices: **front** and **rear**
- Increment the indices as additions and deletions are performed (**rear++** for addition and **front++** for deletion)

(a) `queue.Enqueue('A')`

A				
[0]	[1]	[2]	[3]	[4]

front=0
rear=0

(b) `queue.Enqueue('B')`

A	B			
[0]	[1]	[2]	[3]	[4]

front=0
rear=1

(c) `queue.Enqueue('C')`

A	B	C		
[0]	[1]	[2]	[3]	[4]

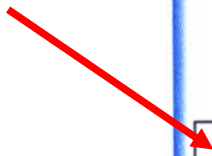
front=0
rear=2

(d) `queue.Dequeue(item)`

	B	C		
[0]	[1]	[2]	[3]	[4]

front=1
rear=2

Dead
space



Implementation Issues

- Maintain two indices: **front** and **rear**
- Make the indices “wrap around” when they reach the end of the array

rear = (rear + 1) % maxQue; //maxQue=size of array

front = (front + 1) % maxQue;

queue.Enqueue('L')

(a) Rear is at the bottom of the array

			J	K
[0]	[1]	[2]	[3]	[4]

front=3
rear=4

(b) Using the array as a circular structure, we can wrap the queue around to the top of the array

L			J	K
[0]	[1]	[2]	[3]	[4]

front=3
rear=0

Implementation Issues

- How do we differentiate between the empty state and the full state?

(a) Initial conditions

		A		
[0]	[1]	[2]	[3]	[4]

front=2
rear=2

(b) `queue.Dequeue(item)`

[0]	[1]	[2]	[3]	[4]

front=3
rear=2

(a) Initial conditions

C	D		A	B
[0]	[1]	[2]	[3]	[4]

front=3
rear=1

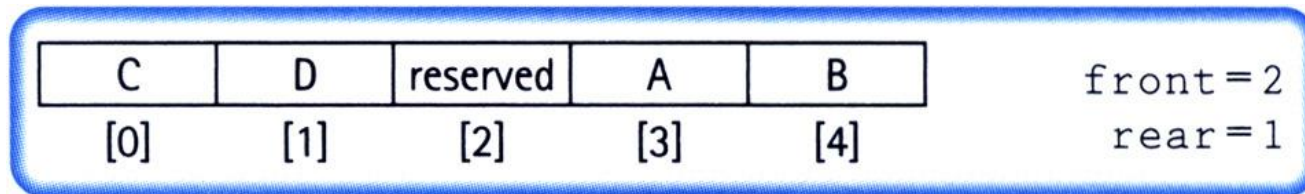
(b) `queue.Enqueue('E')`

C	D	E	A	B
[0]	[1]	[2]	[3]	[4]

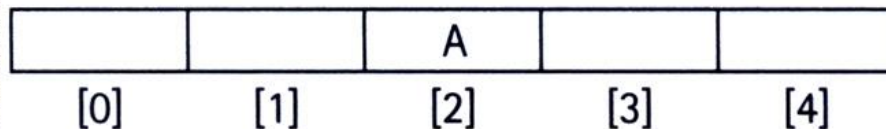
front=3
rear=2

Implementation Issues

- Let front indicate the index of the array slot preceding the front element.
- The array slot preceding the front element is reserved and items are not assigned in that slot.

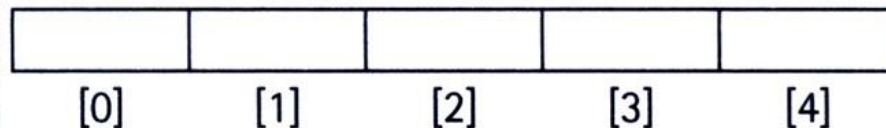


(a) Initial conditions



front = 1
rear = 2

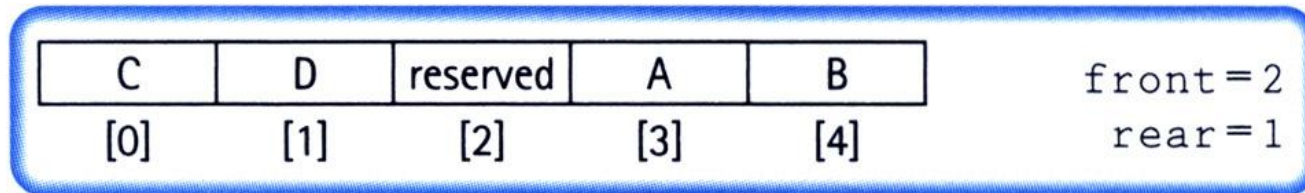
(b) `queue.Dequeue(item)`



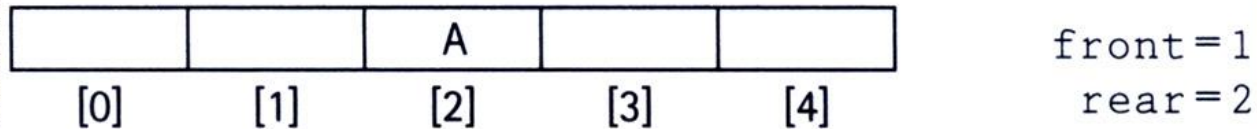
front = 2
rear = 2

Implementation Issues

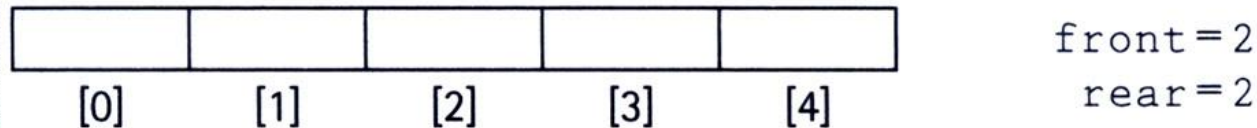
- Full queue when $(\text{rear} + 1) \% \text{maxQue} == \text{front}$
- Empty queue when $\text{front} == \text{rear}$



(a) Initial conditions



(b) queue.Dequeue(item)



queue.h

```
typedef char ItemType;
class FullQueue
{};
class EmptyQueue
{};
class QueType
{
public:
    QueType();
    QueType(int max);
    ~QueType();
    void MakeEmpty();
    bool IsEmpty();
    bool IsFull();
    void Enqueue(ItemType newItem);
    void Dequeue(ItemType& item);
private:
    int front;
    int rear;
    ItemType* items;
    int maxQue;
};
```

queue.cpp

```
#include "Queue.h"

Queue::Queue(int max)
{
    maxQue = max + 1;
    front = maxQue - 1;
    rear = maxQue - 1;
    items = new
ItemType[maxQue];
}

Queue::Queue()
{
    maxQue = 501;
    front = maxQue - 1;
    rear = maxQue - 1;
    items = new
ItemType[maxQue];
}

Queue::~Queue()
{
    delete [] items;
}
```

```
void Queue::MakeEmpty()
{
    front = maxQue - 1;
    rear = maxQue - 1;
}

bool Queue::IsEmpty()
{
    return (rear == front);
}

bool Queue::IsFull()
{
    return ((rear+1)%maxQue == front);
}
```

queuetype.cpp

```
void QueueType::Enqueue(ItemType newItem)
{
    if (IsFull())
        throw FullQueue();
    else
    {
        rear = (rear + 1) % maxQue;
        items[rear] = newItem;
    }
}

void QueueType::Dequeue(ItemType& item)
{
    if (IsEmpty())
        throw EmptyQueue();
    else
    {
        front = (front + 1) % maxQue;
        item = items[front];
    }
}
```

queue.cpp

```
#include "Queue.h"
```

```
Queue::Queue(int max)
{
    maxQue = max + 1;
    front = maxQue - 1;
    rear = maxQue - 1;
    items = new
ItemType[maxQue];
}
```

O(1)

```
Queue::Queue()
{
    maxQue = 501;
    front = maxQue - 1;
    rear = maxQue - 1;
    items = new
ItemType[maxQue];
}
```

O(1)

```
Queue::~Queue()
{
    delete [] items;
}
```

O(1)

```
void Queue::MakeEmpty()
{
    front = maxQue - 1;
    rear = maxQue - 1;
}
```

O(1)

```
bool Queue::IsEmpty()
{
    return (rear == front);
}
```

O(1)

```
bool Queue::IsFull()
{
    return ((rear+1)%maxQue == front);
}
```

O(1)

queuetype.cpp

```
void QueueType::Enqueue(ItemType newItem)
{
    if (IsFull())
        throw FullQueue();
    else
    {
        rear = (rear + 1) % maxQue;
        items[rear] = newItem;
    }
}
```

O(1)

```
void QueueType::Dequeue(ItemType& item)
{
    if (IsEmpty())
        throw EmptyQueue();
    else
    {
        front = (front + 1) % maxQue;
        item = items[front];
    }
}
```

O(1)

An Application of Queue

- You want to form a number **N** as a summation of smaller numbers
- You can only use three prime numbers; **2, 3** and **5**
- You can use any of these three prime numbers as many times as you want
- Can you find the minimum number of prime numbers required to do this?

An Application of Queue

- Say **N** = 11
- Possible solutions
 - 2+2+2+5
 - **3+3+5**
 - 2+2+2+2+3

An Application of Queue

N = 11

queue

[illegible]

cost

[illegible]

An Application of Queue

N = 11

queue

[illegible]

cost

[illegible]

An Application of Queue

N = 11

queue

[illegible]

cost

[illegible]

An Application of Queue

N = 11

queue

[illegible]

cost

[illegible]

An Application of Queue

N = 11

queue

[illegible]

cost

[illegible]

An Application of Queue

N = 11

queue

[illegible]

cost

[illegible]

N = 11

[illegible][illegible]

N = 11

[illegible][illegible]

N = 11

[illegible][illegible]

N = 11

[illegible][illegible]

N = 11

[illegible][illegible]

N = 11

[illegible][illegible]

N = 11

[illegible][illegible]

N = 11

[illegible][illegible]

N = 11

[illegible][illegible]

N = 11

[illegible][illegible]

N = 11

[illegible][illegible]

N = 11

[illegible][illegible]

N = 11

[illegible][illegible]

N = 11

[illegible][illegible]